

VGG16 (2014)

Nếu AlexNet là một cú nổ thì VGG16 chính là biểu tượng của sự đơn giản và đồng nhất, nó được thiết kế để nhận dạng hình ảnh quy mô lớn

1. Triết lý thiết kế: "Sâu hơn nhưng nhỏ hơn"

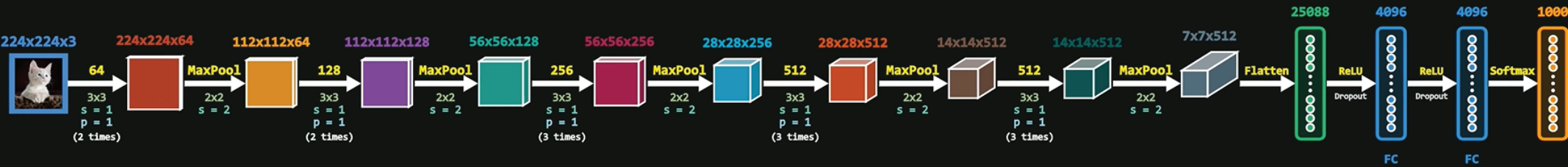
- Vấn đề của AlexNet là dùng các bộ lọc (filter) quá lớn (11x11, 5x5). VGG16 đã thay đổi hoàn toàn cuộc chơi bằng một quy tắc duy nhất đó là chỉ sử dụng bộ lọc 3x3 với bước nhảy (stride) là 1.
- Tại sao lại là 3x3? Tác giả chứng minh rằng việc chồng 3 lớp 3x3 liên tiếp sẽ có "tầm nhìn" tương đương với 1 lớp 7x7, nhưng lại có hai ưu điểm vượt trội:
- Ít tham số hơn: Giúp giảm khối lượng tính toán.
 - Nhiều hàm kích hoạt hơn: Vì mỗi lớp 3x3 đều có ReLU đi kèm, mạng sẽ học được các đặc trưng phi tuyến tính phức tạp hơn.

2. Kiến trúc gồm 16 lớp có trọng số (13 lớp Tích chập + 3 lớp Kết nối đầy đủ)

Cấu trúc của nó giống như một chiếc kim tự tháp ngược: càng vào sâu, hình ảnh càng nhỏ lại nhưng số lượng bộ lọc càng tăng lên

Dưới đây là 5 khối (blocks) chính của VGG16:

- Khối 1 & 2: Sử dụng các bộ lọc 3x3 để trích xuất nét vẽ cơ bản. Số lượng bộ lọc tăng từ 64 lên 128. Sau mỗi khối là một lớp Max Pooling (2x2) để giảm kích thước ảnh đi một nửa.
- Khối 3, 4 & 5: Mỗi khối gồm 3 lớp tích chập liên tiếp. Số lượng bộ lọc tăng lên 256 rồi đạt mức tối đa là 512. Ở giai đoạn này, mạng đã "nhìn" thấy các hình khối rất phức tạp (mắt, mũi, bộ phận cơ thể).
- Các lớp Kết nối đầy đủ (FC): * 3 lớp FC cuối cùng (hai lớp đầu có 4096 nơ-ron, lớp cuối có 1000 nơ-ron cho ImageNet). Đây là "di chứng" từ AlexNet khiến VGG16 cực kỳ nặng nề



VGG16 Architecture (~138M Parameters)

3. Thông số "khủng" của VGG16

Dù kiến trúc rất đẹp và đồng nhất, VGG16 lại là một "gã khổng lồ" về mặt dung lượng:

- Số lượng tham số: Khoảng 138 triệu (Gấp đôi AlexNet).
- Dung lượng file: Một model VGG16 đã huấn luyện xong có thể nặng tới hơn 500MB.
- Chi phí tính toán: Rất chậm khi huấn luyện vì số lượng phép tính khổng lồ ở các lớp tích chập phía sau (với 512 bộ lọc).

4. Những điều làm nên tên tuổi của VGG16

1. Triết lý "Nhỏ mà có võ" (Small Filters)
 - Thay vì dùng bộ lọc lớn, VGG16 chỉ dùng duy nhất bộ lọc 3x3
 - Lợi ích: Bạn có 3 lần dùng hàm kích hoạt ReLU thay vì 1 lần, giúp mạng học được các đường cong và chi tiết cực kỳ phức tạp (phi tuyến tính cao hơn).
2. Tính đồng nhất (Uniformity)
 - Kiến trúc VGG cực kỳ dễ nhớ: Tất cả các lớp Convolution đều là 3x3, Stride = 1, Padding = 1. Tất cả các lớp Max Pooling đều là 2x2, Stride = 2.
 - Cứ sau mỗi lớp Pooling, số lượng bộ lọc lại gấp đôi (64 -> 128 -> 256 -> 512). Sự đồng nhất này biến VGG16 thành "mẫu chuẩn" để các nhà nghiên cứu sau này copy và phát triển.
3. Khả năng "Học chuyển đổi" (Transfer Learning) xuất sắc
 - VGG16 trích xuất đặc trưng ảnh (feature extraction) cực tốt. Cho đến tận ngày nay, khi làm các bài toán mới mà không có nhiều dữ liệu, người ta thường lấy "bộ não" đã huấn luyện sẵn của VGG16, cắt bỏ phần đuôi và lắp phần mới vào. Nó hoạt động ổn định đến kinh ngạc

5. Vấn đề (Những gánh nặng "nghìn cân")

1. "Cỗ máy" ngốn tài nguyên (Too Heavy): VGG16 có tới 138 triệu tham số. Để bạn dễ hình dung:
 - AlexNet: 60 triệu tham số.
 - ResNet-50 (sâu hơn nhiều): Chỉ khoảng 25 triệu tham số.
 - Hệ quả: File model của VGG16 nặng hơn 500MB, cực kỳ khó cài đặt lên các thiết bị di động hay camera giám sát vốn có bộ nhớ hạn chế
2. Nút thắt cổ chai ở các lớp Kết nối đầy đủ (FC Layers)
 - Khoảng 90% tham số của VGG16 nằm ở 3 lớp FC cuối cùng
 - Các lớp này không giúp ích nhiều cho việc nhận diện hình dạng nhưng lại chiếm gần hết bộ nhớ và làm chậm tốc độ xử lý
3. Vấn đề Hội tụ và Biến mất Gradient
 - Loss không giảm
 - Accuracy không tăng (thậm chí giảm): Nghe vô lý đúng không? -> Thêm layer mà model lại học kém hơn
 - Khi mạng sâu hơn nữa (như VGG19), việc huấn luyện trở nên cực kỳ khó khăn vì tín hiệu lỗi bị nhỏ dần khi truyền về các lớp đầu (Vanishing Gradient) -> Thông tin không truyền được từ output -> input
4. Tốc độ rùa bò
 - Vì có quá nhiều phép tính tích chập với số lượng bộ lọc lớn (512), VGG16 chạy rất chậm trên các dòng GPU phổ thông so với các kiến trúc hiện đại có cùng độ chính xác